FACE AND IDENTITY VERIFICATION USING CLOUD BASED SAAS MODELS

by

Ahmet Abdullah Gültekin - 150121025

Ayşe Gülsüm Eren - 150120005

Ayşenur Arıcı - 150123825

CSE4297 Engineering Project 1

Analysis and Design Document

Supervised by:

Assoc. Prof. Dr. Mustafa Ağaoğlu

Marmara University, Faculty of Engineering

Computer Engineering Department

25.01.2026

Copyright © Group members listed above, 2026. All rights reserved.

Table of Contents

1. Introduction 3

1.1 Problem Description **3**

1.2 Scope **3**

1.3 Definitions and Acronyms **5**

2. Literature Survey 5

2.1 Identity and Access Management (IAM) Systems **6**

2.2 Deep Learning Based Face Recognition Technologies **6**

2.3 Liveness Detection and Anti-Spoofing **7**

2.4 Identity Document Verification and Standarts **8**

2.5 Industrial Biometric Authentication Solutions **9**

2.6 Cloud-Native Biometric Systems and Multi-Tenant Architectures **9**

2.7 Positioning and Differentiation of the Proposed System **10**

3. Project Requirements 10

3.1 Functional Requirements **10**

3.2 Non-Functional Requirements **10**

4. System Design 18

4.1 Use Case Diagrams **18**

4.2 Class and ER Diagrams **24**

4.3 User Interface Design **27**

4.4 Test Plan **29**

5. Software Architecture 31

5.1 Architectural Style **32**

5.2 Component Architecture **32**

5.3 Data Architecture **36**

5.4 Deployment Architecture **37**

6. Tasks Accomplished 39

6.1 Current State of the System **39**

6.2 Task Log **42**

6.3 Task Plan **42**

1. Introduction

1.1 Problem Description

Traditional authentication methods present significant security vulnerabilities and poor user experiences in modern digital and physical access scenarios. According to Verizon's 2024 Data Breach Investigations Report [1], **81% of hacking-related breaches involve stolen or weak passwords**, while the Identity Theft Resource Center reports a **78% increase in data compromise events** [2] between 2022-2023 affecting over 350 million individuals. Simultaneously, physical access cards can be cloned with readily available $50 RFID readers, and passive biometric systems remain vulnerable to spoofing attacks using static photos (success rate: 67% against basic systems), videos, or sophisticated 3D masks.

This project addresses these challenges by developing **FIVUCSAS** (Face and Identity Verification Using Cloud-based SaaS) - a multi-tenant, cloud-native biometric authentication platform. The system integrates advanced face recognition with an innovative active liveness detection algorithm called "Biometric Puzzle" to provide robust protection against fraud while maintaining excellent user experience through developer-friendly APIs.

The platform targets B2B and B2B2C markets, unifying both physical access control (door entry, kiosk authentication) and digital authentication (system login, transaction verification) under a single identity management solution.

1.2 Scope

1.2.1 In Scope

Backend Services: The system utilizes a microservices architecture comprising the Identity Core API developed with Spring Boot 3.2 (Java 21) for business logic and the Biometric Processor API built on FastAPI (Python 3.11) for AI-driven operations.

Client Applications: The platform supports multiple interfaces including a Web Admin Dashboard (React 18), a native Mobile App (Android), and a cross-platform Desktop App (Windows/Linux/macOS) developed using Kotlin Multiplatform.

Database Architecture: Data persistence is managed via a multi-tenant PostgreSQL 16 database, utilizing the pgvector extension to efficiently store and query high-dimensional face embeddings [3].

Face Biometrics: The system performs Face Recognition (1:N) and Face Verification (1:1) tasks utilizing the DeepFace [4] library, offering flexibility with 9 different model options.

Liveness Detection: A hybrid approach combines the "Biometric Puzzle" active liveness challenge with passive anti-spoofing techniques, powered by MediaPipe [5] to track 468 facial landmarks.

Infrastructure: Deployment is orchestrated using Docker Compose for containerization, with a NGINX [12] API Gateway handling request routing and Redis [6] employed for high-performance caching.

Security: The platform enforces strict security measures including JWT-based authentication, Role-Based Access Control (RBAC), BCrypt for password hashing, and API rate limiting.

1.2.2 Out of Scope

Additional Biometric Modalities: Although the system's hexagonal architecture [7] is designed for extensibility, the integration of fingerprint, voice, or iris recognition is deferred. This decision allows the team to concentrate on perfecting facial recognition and liveness detection algorithms, avoiding the deployment complexity associated with specialized hardware peripherals like fingerprint scanners.

Production-Grade Orchestration: While the system is cloud-native, complex container orchestration via Kubernetes and Helm charts is excluded from the current scope. Docker Compose [8] is utilized for all development and demonstration purposes, providing a sufficient environment for validating the microservices architecture without the operational overhead of a full production cluster.

Hardware Manufacturing: The project focuses strictly on software engineering principles; therefore, the design and manufacturing of physical access control hardware are excluded. Edge device interactions are simulated through software interfaces, though the system remains compatible with standard commercial hardware such as Raspberry Pi-based units.

Comprehensive Billing Integration: To maintain focus on core biometric innovations, advanced subscription management and third-party payment gateways (e.g., Stripe) are not implemented. Multi-tenancy is enforced through database-level quotas, which is sufficient to demonstrate the platform's SaaS capabilities without implementing generic billing logic.

1.2.3 Constraints

Technology: The project exclusively utilizes open-source technologies with permissive licenses, strictly excluding proprietary databases, machine learning models, or commercial frameworks.

Infrastructure: Development is primarily conducted on local environments utilizing Docker Compose [8]. VPS hosting is permitted solely for demonstration and testing purposes, not for production deployment.

Hardware: Liveness detection accuracy is constrained by the client device's hardware, specifically requiring a recommended minimum camera resolution of 720p. System inference is optimized for CPU execution, with GPU acceleration considered optional for scalability.

Timeline: The project scope is bound by the academic semester constraints (September 2025 - January 2026), necessitating the prioritization of core biometric features over auxiliary functionalities.

1.3 Definitions and Acronyms

Term	Definition
ADD	Architectural Design Document
API	Application Programming Interface
BCrypt	Password hashing algorithm with configurable work factor
DeepFace	Python library providing face recognition models
EAR	Eye Aspect Ratio - metric for blink detection
Embedding	High-dimensional vector representation of a face
FAR	False Acceptance Rate
FRR	False Rejection Rate
GDPR	General Data Protection Regulation (EU)
Hexagonal Architecture	Ports and Adapters pattern for clean separation of concerns
JWT	JSON Web Token for stateless authentication
KMP	Kotlin Multiplatform - cross-platform development framework
KVKK	Turkish Personal Data Protection Law (No. 6698)
MAR	Mouth Aspect Ratio - metric for smile detection
MediaPipe	Google's ML library for real-time face landmark detection
MVP	Minimum Viable Product
pgvector	PostgreSQL extension for vector similarity search
RBAC	Role-Based Access Control
SaaS	Software as a Service
Spoofing	Attempt to bypass biometric authentication using fake biometrics

Table 1: *Definitions and Acronyms*

2. Literature Survey

This section provides a comprehensive review of academic and industrial research related to biometric authentication systems, with a particular focus on Identity and Access Management (IAM), deep learning–based face recognition, liveness detection, identity document verification, and cloud-native multi-tenant architectures. The objective is to identify limitations in existing approaches and to position the proposed system within the broader biometric and IAM landscape.

2.1 Identity and Access Management (IAM) Systems

Identity and Access Management (IAM) systems form the foundation of secure digital services by regulating authentication, authorization, and user lifecycle management. Leading commercial IAM platforms such as Okta, Auth0, and Microsoft Azure Active Directory have established mature ecosystems centered on traditional authentication paradigms. These platforms primarily employ password-based authentication, multi-factor authentication (MFA) using One-Time Passwords (OTP), push-based approval mechanisms, and device-bound biometric authentication such as Apple Face ID and Android fingerprint recognition.

Despite their widespread adoption, existing IAM solutions exhibit several structural limitations. First, device-bound biometric mechanisms restrict identity portability by tightly coupling authentication to specific hardware ecosystems, resulting in fragmented user experiences. Second, physical access control systems such as doors, turnstiles, and kiosks are typically managed through isolated third-party integrations, leading to disconnected physical and digital identity domains. Finally, many IAM platforms treat biometrics as a supplementary authentication factor rather than a core identity primitive, limiting their resistance to advanced spoofing and replay attacks.

Recent academic and industrial studies emphasize the growing need for IAM systems that unify physical and digital identity under a single trust framework, support strict tenant isolation in shared environments, and integrate advanced biometric verification pipelines. These requirements motivate the adoption of cloud-native, biometric-first IAM architectures capable of scaling across organizations while preserving security, privacy, and regulatory compliance.

2.2 Deep Learning Based Face Recognition Technologies

Face recognition has undergone a significant transformation with the adoption of deep learning–based representation learning. Early face recognition systems relied on handcrafted features such as Local Binary Patterns (LBP) and Histogram of Oriented Gradients (HOG), which were highly sensitive to variations in pose, illumination, and occlusion. The introduction of deep convolutional neural networks (CNNs) enabled end-to-end learning of discriminative facial embeddings, dramatically improving robustness and accuracy.

Pioneering work such as **DeepFace** demonstrated that deep neural networks could approach human-level performance in face verification by learning high-dimensional facial representations. **FaceNet** [11] further advanced the field by introducing a unified embedding space optimized using triplet loss, enabling direct comparison of facial identities through simple distance metrics.

Subsequent research focused on enhancing inter-class separability and intra-class compactness through margin-based loss functions. **ArcFace** [10] introduced additive

angular margin loss, achieving state-of-the-art performance on benchmarks such as LFW. Extensions including **CosFace**, **SphereFace**, **MagFace**, and **AdaFace** incorporated cosine margins, angular normalization, and quality-aware weighting strategies to improve robustness under low-resolution imaging, motion blur, and off-angle facial captures.

While these models achieve exceptional accuracy in controlled benchmark evaluations, most studies focus primarily on algorithmic performance. System-level concerns such as scalability, latency, multi-tenant isolation, and operational monitoring are often outside the scope of academic research. As a result, a gap remains between research-grade face recognition models and production-ready biometric platforms.

To bridge this gap, unified frameworks such as **DeepFace** [18] **(Serengil & Ozpinar)** provide standardized access to multiple state-of-the-art face recognition models, abstracting face detection, alignment, embedding generation, and similarity computation. Such frameworks enable the practical deployment of advanced face recognition models in real-world systems while preserving architectural flexibility.

Table 2 summarizes representative deep learning–based face recognition models, highlighting their embedding dimensionality, underlying network architectures, and reported performance on the Labeled Faces in the Wild (LFW) benchmark.

Model	Embedding Dimension	Architecture	LFW Accuracy
VGG-Face	2,622	VGGNet	98.95%
Facenet512	512	Inception ResNet	99.65%
ArcFace	512	ResNet	99.82%
DeepFace	4,096	AlexNet	97.35%
OpenFace	128	Inception	92.92%

Table 2: *Comparison of Deep Learning–Based Face Recognition Models*

The comparison highlights a clear trade-off between embedding dimensionality and recognition accuracy. While earlier architectures such as DeepFace rely on high-dimensional feature vectors, more recent margin-based models such as ArcFace achieve superior accuracy with significantly smaller embeddings. This reduction in dimensionality is particularly advantageous for large-scale biometric systems, as it improves storage efficiency and accelerates similarity search operations in production environments.

2.3 Liveness Detection and Anti-Spoofing

Liveness detection has become a critical component of biometric authentication systems due to the increasing prevalence of presentation attacks, including printed photographs, replayed videos, and high-quality deepfake content. Face anti-spoofing techniques are commonly categorized into passive, active, and hybrid approaches , as standardized in ISO/IEC 30107-3 [9].

Passive liveness detection methods analyze visual cues from single images or video streams without requiring user interaction. Common techniques include texture analysis using Local Binary Patterns, frequency-domain analysis for detecting moiré artifacts, color space consistency checks, and CNN-based classifiers trained on spoofing datasets. Although passive approaches offer a seamless user experience, they remain vulnerable to high-quality attacks, three-dimensional masks, and generative AI–based spoofing.

Active liveness detection introduces challenge–response mechanisms that require deliberate user actions, such as blinking, smiling, head movements, or gaze tracking. Quantitative metrics such as Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR) are widely used to detect these actions. The emergence of real-time facial landmark frameworks such as **Google MediaPipe Face Mesh**, which provides 468 facial landmarks, has significantly improved the feasibility of on-device active liveness verification.

Hybrid approaches combine passive and active techniques to maximize robustness against both traditional and AI-generated attacks. Recent literature highlights that hybrid pipelines achieve superior security while maintaining acceptable latency and usability. However, many proposed solutions remain model-centric and lack integration into complete, end-to-end authentication workflows.

The increasing sophistication of generative adversarial attacks underscores the need for liveness mechanisms that are dynamic, randomized, and context-aware—capabilities that static classifiers alone cannot reliably provide.

2.4 Identity Document Verification and Standarts

In high-assurance identity verification scenarios, biometric authentication is frequently complemented by identity document verification. Research in this area includes ID–selfie matching pipelines, optical character recognition (OCR), Machine Readable Zone (MRZ) parsing, and document layout analysis. Systems such as **DocFace** demonstrate that ID-specific face matching models outperform generic face recognition approaches when comparing document photographs with live facial captures.

International standards play a crucial role in document-based identity verification. The **International Civil Aviation Organization (ICAO)** defines specifications for electronic Machine-Readable Travel Documents (eMRTDs), including secure storage of facial biometric data in NFC-enabled passport chips. NFC-based verification workflows provide strong cryptographic guarantees and are widely adopted in border control and e-government systems.

Commercial solutions such as **BioGATE Pass** leverage ICAO-compliant NFC pipelines to achieve high levels of trust. However, these systems are often tightly coupled to specialized hardware and lack extensibility for broader cloud-based SaaS deployments.

Consequently, recent academic literature increasingly emphasizes modular and extensible architectures that allow document verification components to coexist with biometric and behavioral authentication modules.

2.5 Industrial Biometric Authentication Solutions

Several industrial platforms provide large-scale face recognition and liveness detection services. **Microsoft Azure Face Liveness** and **Amazon Rekognition** offer highly accurate, standards-compliant biometric APIs with global availability. However, these platforms are fully proprietary, cloud-dependent, and provide limited transparency, customization, and control over data processing. Privacy-sensitive organizations and academic institutions often face regulatory, cost, and vendor lock-in concerns when adopting such services.

Regional providers such as **Sodec Technologies** focus primarily on e-government and banking applications but typically lack multi-tenant SaaS architectures and advanced active liveness detection mechanisms. Open-source solutions such as **FaceAuth Vision** support offline and edge deployments but fall short in modern anti-spoofing robustness and enterprise-grade scalability.

Comparative analyses indicate that no single existing solution simultaneously provides open extensibility, hybrid liveness detection, cloud-native scalability, and unified management of physical and digital identities.

2.6 Cloud-Native Biometric Systems and Multi-Tenant Architectures

Recent research emphasizes the importance of cloud-native architectural principles such as microservices, containerization, and event-driven communication for scaling biometric workloads. Vector databases such as pgvector [3] and similarity search libraries like FAISS [13] enable efficient storage and retrieval of high-dimensional facial embeddings, supporting large-scale identification and verification scenarios.

Multi-tenant architectures introduce additional challenges, including strict data isolation, tenant-specific configuration, and performance fairness under shared infrastructure. While numerous studies address individual aspects of biometric scalability or tenant isolation, relatively few systems integrate identity management, biometric processing, and liveness detection within a single cohesive architectural framework.

Modern architectural patterns such as **Hexagonal Architecture** and **Clean Architecture** are increasingly recommended to decouple core business logic from infrastructure concerns, enabling maintainability, testability, and long-term system evolution.

2.7 Positioning and Differentiation of the Proposed System

The reviewed literature reveals a clear gap between state-of-the-art biometric research and deployable, multi-tenant IAM platforms. Existing solutions either focus narrowly on algorithmic innovation or offer closed, inflexible commercial systems with limited extensibility.

The proposed system addresses this gap by integrating advanced deep learning–based face recognition models, hybrid liveness detection, and identity document verification within a cloud-native SaaS architecture. By combining active challenge–response liveness mechanisms with configurable multi-model face recognition pipelines, the platform achieves strong resistance to spoofing while maintaining usability and scalability. Its microservices-based design enables unified management of physical and digital access, tenant-level customization, and extensibility toward future biometric modalities.

In this context, the system positions itself as a bridge between academic biometric research and practical, enterprise-grade identity platforms, aligning with contemporary trends in secure, scalable, and privacy-aware authentication systems.

3. Project Requirements

3.1 Functional Requirements

The following functional requirements are described at an abstract level to emphasize system behavior and constraints, while implementation-specific details are presented in the system design and architecture sections.

ID	Functional Requirements	Description
FR-1	Identity & Access Management	The system shall authenticate users and manage accounts securely within tenant boundaries.
FR-2	Biometric Enrollment	The system shall enroll facial biometric data after validating quality and liveness.
FR-3	Biometric Verification	The system shall verify user identity using facial biometrics with optional liveness checks.
FR-4	Multi-Tenant Management	The system shall support isolated tenant configuration, quotas, and settings.

FR-5	Authorization & RBAC	The system shall enforce role-based access control for all protected operations.
FR-6	Auditing & Compliance	The system shall record and expose audit logs and verification history securely.

Table 3: Functional Requirements (FR)

3.1.1 Identity and Access Management (FR-1)

3.1.1.1 Description

The system shall provide a secure mechanism for authenticating users and managing administrative accounts within strictly isolated tenant boundaries.

3.1.1.2 Inputs

- User credentials (email address, password)
- Tenant identifiers
- Session-related information (IP address, User-Agent, timestamp)

3.1.1.3 Processing

1. Validate credential formats (RFC 5322 for email, minimum password requirements)

2. Enforce user uniqueness within each tenant namespace

3. Verify password against stored BCrypt hash (work factor 12)

4. Generate signed JWT access token (15-minute expiry) and UUID-based refresh token (7-day expiry)

5. Store session metadata in Redis cache

3.1.1.4 Outputs

- Authentication tokens (Access Token, Refresh Token)
- User profile information (ID, email, roles, tenant context)
- Session expiration timestamps

3.1.1.5 Error/Data Handling

- Invalid Credentials: HTTP 401 Unauthorized with generic error message
- Duplicate Account: HTTP 409 Conflict during registration
- Locked Account: HTTP 423 Locked after 5 consecutive failed attempts
- Invalid Token: HTTP 401 Unauthorized for expired or malformed tokens

3.1.2 Biometric Enrollment (FR-2)

3.1.2.1 Description

The system shall support the secure enrollment of facial biometric data after verifying both image quality and user liveness.

3.1.2.2 Inputs

- Facial images captured during interactive liveness challenge
- User identifier (from authenticated session)
- Tenant context

3.1.2.3 Processing

1. Detect facial region using configurable backend (RetinaFace, MTCNN, or MediaPipe)

2. Assess biometric sample quality (brightness > 0.3, sharpness > 0.5, frontal pose within 15°)

3. Validate liveness through 'Biometric Puzzle' challenge sequence

4. Generate standardized biometric embedding using selected model (default: Facenet512)

5. Store embedding vector in PostgreSQL with pgvector extension

3.1.2.4 Outputs

- Unique enrollment identifier (UUID)
- Quality confidence score (0.0 - 1.0)
- Enrollment status (SUCCESS, PENDING, FAILED)

3.1.2.5 Error/Data Handling

- No Face Detected: HTTP 400 Bad Request with guidance message
- Image Quality Insufficient: HTTP 422 Unprocessable Entity with specific quality metrics
- Liveness Validation Failed: HTTP 403 Forbidden with failure reason (e.g., 'Blink not detected')

3.1.3 Biometric Verification (FR-3)

3.1.3.1 Description

The system shall verify user identity by comparing live facial biometric samples against previously enrolled biometric templates, with optional liveness validation.

3.1.3.2 Inputs

- Live facial image (base64 encoded or multipart upload)
- User or tenant context for template retrieval
- Optional: Liveness challenge response

3.1.3.3 Processing

1. Generate biometric embedding from input image using configured model

2. Retrieve enrolled reference template(s) from vector database

3. Compute cosine similarity score between input and reference embeddings

4. Compare similarity score against configurable threshold (default: 0.6)

5. Log verification attempt with result and metadata

3.1.3.4 Outputs

- Verification decision (MATCH / NO_MATCH)
- Similarity confidence score (0.0 - 1.0)
- Matched identity information (if successful)

3.1.3.5 Error/Data Handling

- No Enrollment Found: HTTP 404 Not Found
- Verification Failed: HTTP 401 Unauthorized when score < threshold
- Rate Limit Exceeded: HTTP 429 Too Many Requests

3.1.4 Multi-Tenant Management (FR-4)

3.1.4.1 Description

The system shall support isolated configurations for multiple tenants, enabling independent management of quotas, limits, and operational settings.

3.1.4.2 Inputs

- Tenant metadata (organization name, domain, contact email)
- Configuration parameters (max users, enrollment quota, API rate limits)
- Billing tier selection

3.1.4.3 Processing

1. Validate tenant identifier uniqueness across the platform

2. Provision tenant-specific configuration with default quotas

3. Generate tenant API credentials (API Key, Secret)

4. Apply row-level security policies for data isolation

3.1.4.4 Outputs

- Unique tenant identifier (UUID)
- Tenant API credentials
- Configuration confirmation with applied quotas

3.1.4.5 Error/Data Handling

- Duplicate Tenant: HTTP 409 Conflict

- Quota Exceeded: HTTP 429 Too Many Requests when tenant limits are reached
- Invalid Configuration: HTTP 422 Unprocessable Entity for invalid parameters

3.1.5 Authorization & Role Based Access Control (FR-5)

3.1.5.1 Description

The system shall enforce role-based access control to regulate access to protected resources based on assigned permissions.

3.1.5.2 Inputs

- Requesting user identity (extracted from JWT)
- Target resource identifier (URI path)
- Requested operation (HTTP method: GET, POST, PUT, DELETE)

3.1.5.3 Processing

1. Decode and validate JWT token signature and expiration

2. Extract user roles from token claims

3. Resolve role-to-permission mappings from RBAC configuration

4. Evaluate if user permissions include the requested operation on the target resource

5. Cache permission decisions for performance (TTL: 5 minutes)

3.1.5.4 Outputs

- Authorization decision (ALLOW / DENY)
- Effective permissions list (for debugging/audit)

3.1.5.5 Error/Data Handling

- Insufficient Permissions: HTTP 403 Forbidden
- Invalid Token: HTTP 401 Unauthorized
- Resource Not Found: HTTP 404 Not Found

3.1.6 Auditing & Compliance (FR-6)

3.1.6.1 Description

The system shall record and expose immutable audit logs and verification histories to support security monitoring and compliance with applicable data protection regulations (GDPR, KVKK).

3.1.6.2 Inputs

- User action details (actor, action type, target resource)
- Request context (IP address, User-Agent, timestamp)

- Affected data identifiers

3.1.6.3 Processing

1. Sanitize input data to exclude sensitive credentials and biometric data

2. Append ISO 8601 timestamp and tenant context

3. Generate immutable audit record with unique event ID

4. Write to append-only audit log storage with tenant isolation

3.1.6.4 Outputs

- Paginated audit record list with filtering support
- Verification history reports (exportable as CSV/JSON)
- Compliance summary dashboards

3.1.6.5 Error/Data Handling

- Access Denied: HTTP 403 Forbidden (only Tenant Admin and above)
- Invalid Filter Parameters: HTTP 400 Bad Request
- Export Limit Exceeded: HTTP 429 Too Many Requests for large exports

3.2 Non-Functional Requirements

Category	Requirement	Measure / Verification
Performance	The system shall respond to requests within acceptable latency.	95th percentile latency < 200ms for authentication; < 500ms for biometric verification.
Scalability	The system shall handle concurrent users without degradation.	Load testing results showing stability up to 100 concurrent users per instance.
Reliability	The system shall remain available during operational hours.	Targeted uptime of 99.5% with an RPO of < 1 hour for backups.
Security	The system shall protect data against unauthorized access.	TLS 1.3 encryption for transmission and AES-256 encrypted storage for embeddings.
Usability	Users shall complete enrollment with minimal effort.	Total enrollment time < 60 seconds including liveness challenges.
Maintainability	The system shall be modular and easy to extend.	Use of Hexagonal Architecture and > 70% unit test coverage.
Portability	The system shall run consistently across platforms.	Docker containerization and KMP shared code verification.

Table 4: Non-Functional Requirements (NFR)

The following requirements specify the operational criteria and constraints that define the system's quality and performance. These are categorized by their specific attributes to ensure the platform is robust, secure, and maintainable.

3.2.1 Performance

The system shall provide responsive system behavior for authentication and biometric operations in order to support real-time usage scenarios such as physical access control, kiosk authentication, and online identity verification. Performance requirements are critical due to the computational nature of biometric processing and the user-facing interaction model of the system. The platform shall ensure that authentication, enrollment, and verification workflows complete within acceptable time bounds so that biometric verification does not introduce perceptible delays or degrade user experience. Performance compliance shall be evaluated through controlled testing under representative workloads.

3.2.2 Scalability

The system shall be designed to scale horizontally in order to support multiple tenants, increasing user populations, and growing biometric workloads. As a Software-as-a-Service platform targeting B2B and B2B2C use cases, the system must accommodate variations in tenant size and request volume without requiring architectural redesign. Scalability requirements ensure that biometric enrollment and verification services can handle concurrent requests across tenants while preserving isolation and performance. Scalability shall be validated through load and stress testing aligned with projected usage scenarios.

3.2.3 Reliability

The system shall operate reliably during designated operational periods, ensuring consistent availability of identity verification services. Reliability is a critical requirement due to the system's role in security-sensitive workflows, where service interruptions may block user access or disrupt operations. The system shall tolerate partial failures and recover gracefully without cascading effects across components or tenants. Reliability targets shall be assessed through availability monitoring, fault injection, and controlled failure scenarios.

3.2.4 Security

The system shall enforce strict security controls to protect identity information and biometric data throughout their lifecycle. Security requirements address confidentiality, integrity, and access control for sensitive data, particularly given the irreversible nature

of biometric identifiers. The system shall prevent unauthorized access, ensure strong isolation between tenants, and minimize exposure of biometric data by operating on derived representations rather than raw biometric artifacts. Compliance with security requirements shall be verified through security testing, auditability, and adherence to best practices in secure system design.

3.2.5 Usability

The system shall provide an intuitive and efficient user experience across all supported client applications, including web, mobile, and desktop environments. Usability requirements are especially important for biometric interactions, where unclear feedback or complex flows can lead to user frustration or enrollment failure. The system shall guide users through enrollment and verification processes with clear instructions and meaningful feedback, enabling successful completion with minimal effort. Usability effectiveness shall be evaluated based on task completion success and consistency of interaction flows.

3.2.6 Maintainability

The system shall be maintainable and extensible to support future enhancements, algorithm upgrades, and evolving security requirements. Given the rapid advancement of biometric technologies and identity verification standards, the system must allow components to be modified or replaced without disrupting core functionality. Maintainability requirements emphasize modular structure, clear separation of concerns, and ease of testing, enabling the system to evolve throughout its lifecycle. Compliance with maintainability goals shall be assessed through architectural conformance and codebase organization.

3.2.7 Portability

The system shall operate consistently across supported deployment environments and client platforms. Portability requirements ensure that the system can be deployed in different infrastructure settings and accessed through multiple operating systems without behavioral inconsistencies. This is essential for a SaaS platform intended for integration into diverse organizational environments. Portability shall be validated through deployment and execution testing across supported platforms and environments.

4. System Design

4.1 Use Case Diagrams

4.1.1 System Use Cases by Actor

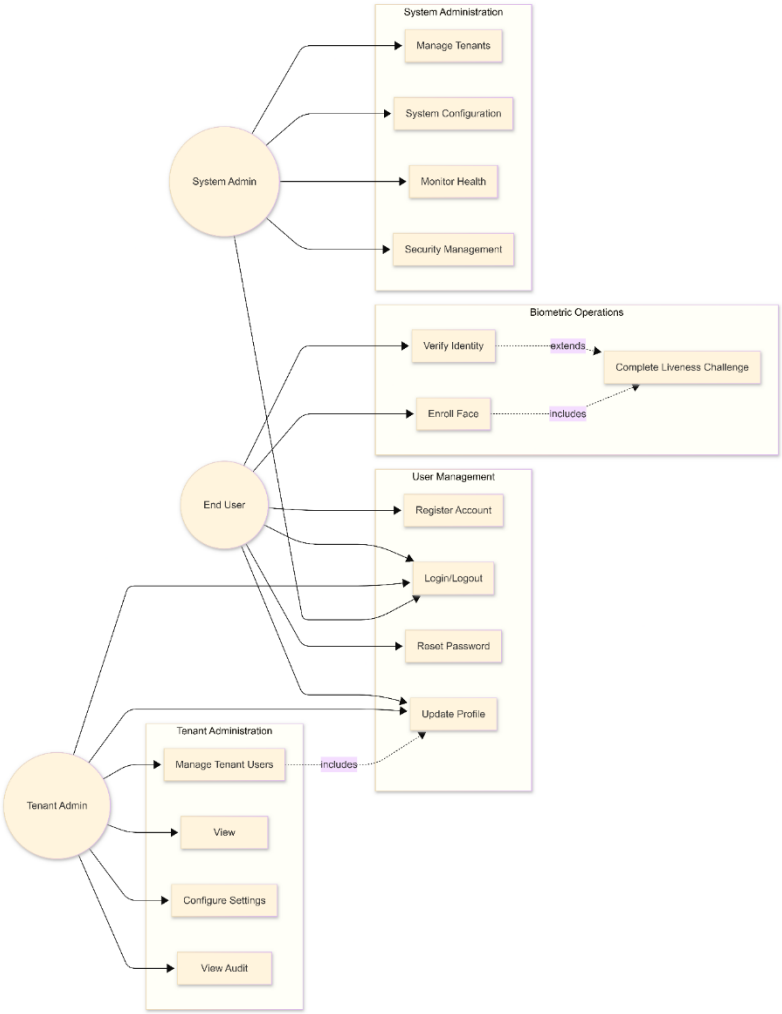


Figure 1: Use case diagram illustrating role-based system interactions. The diagram shows the responsibilities of System Admin, Tenant Admin, and End User, including system administration, tenant management, user account operations, and biometric processes. Biometric enrollment and identity verification are secured through a liveness challenge, modeled using include and extend relationships.

4.1.2 Face Enrollment Use Case

Use case diagram illustrating the face enrollment process in FIVUCSAS system. The main flow includes authentication verification, liveness detection through biometric challenges, face capture, quality assessment, and embedding generation. Alternative flows handle duplicate enrollment, poor image quality, and liveness verification failures with appropriate retry mechanisms.

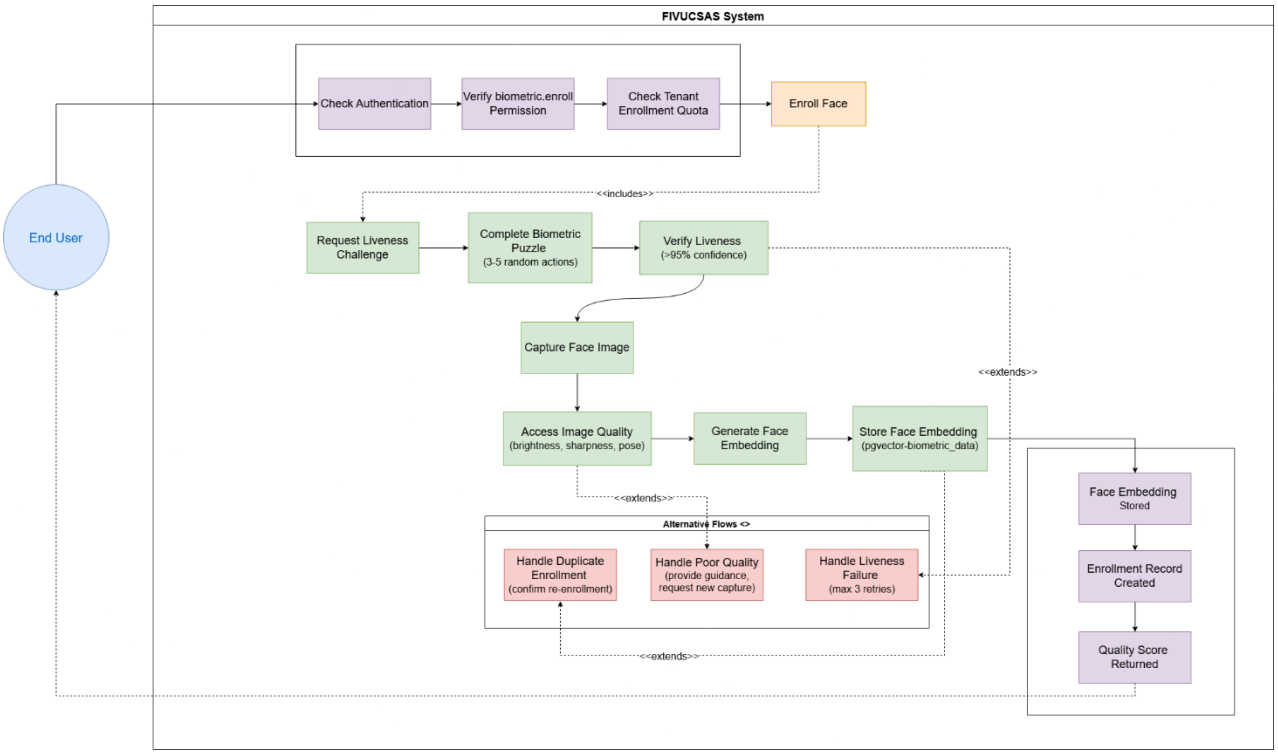


Figure 2:Face Enrollment Use Case Diagram

4.1.3 Face Verification Use Case

Use case diagram illustrating the identity verification process in FIVUCSAS system. The main flow includes request submission, rate limit validation (token bucket), face detection, embedding generation, enrolled embedding retrieval, cosine similarity calculation, threshold comparison (default: 0.6), and return result. Exception scenarios handle rate limit exceeded (429), no face detected (400), and below-threshold score conditions.

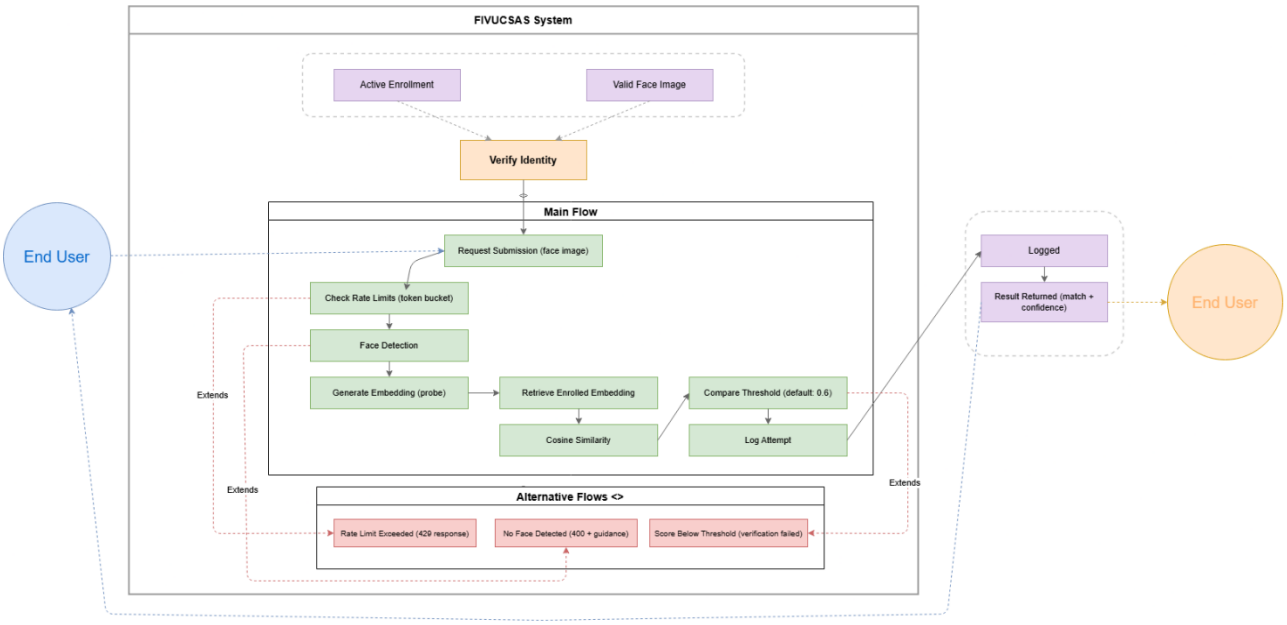


Figure 3: Face Verification Use Case Diagram

4.1.4 Sequence Diagrams

4.1.4.1 User Registration Sequence

Sequence diagram illustrating the user registration flow in FIVUCSAS system. The main flow includes: registration request submission, RFC 5322 email format validation, tenant validation, duplicate email verification, BCrypt password hashing, user record creation, JWT access token generation , UUID-based refresh token generation, session storage in Redis, and successful registration response.

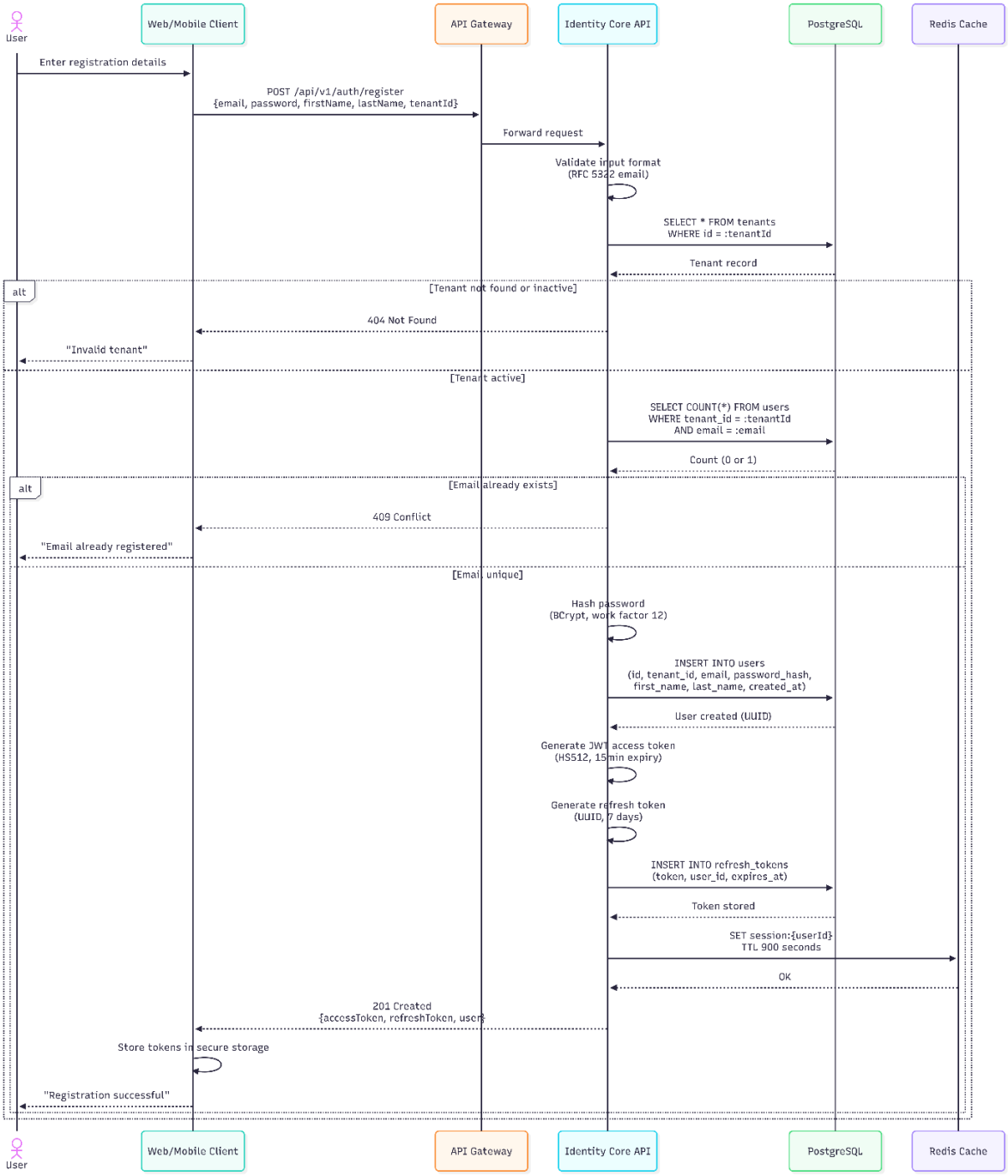


Figure 4: User Registration Sequence Diagram

4.1.4.2 Biometric Enrollment with Liveness Sequence

The diagram illustrates interactions between six components: User, Biometric Processor, Identity Core API, DeepFace Library, MediaPipe, and PostgreSQL. The main flow includes liveness challenge initiation, liveness verification through eye blink and head movement detection, high-quality face image capture after successful verification, face detection, quality analysis, embedding vector generation, and database storage.

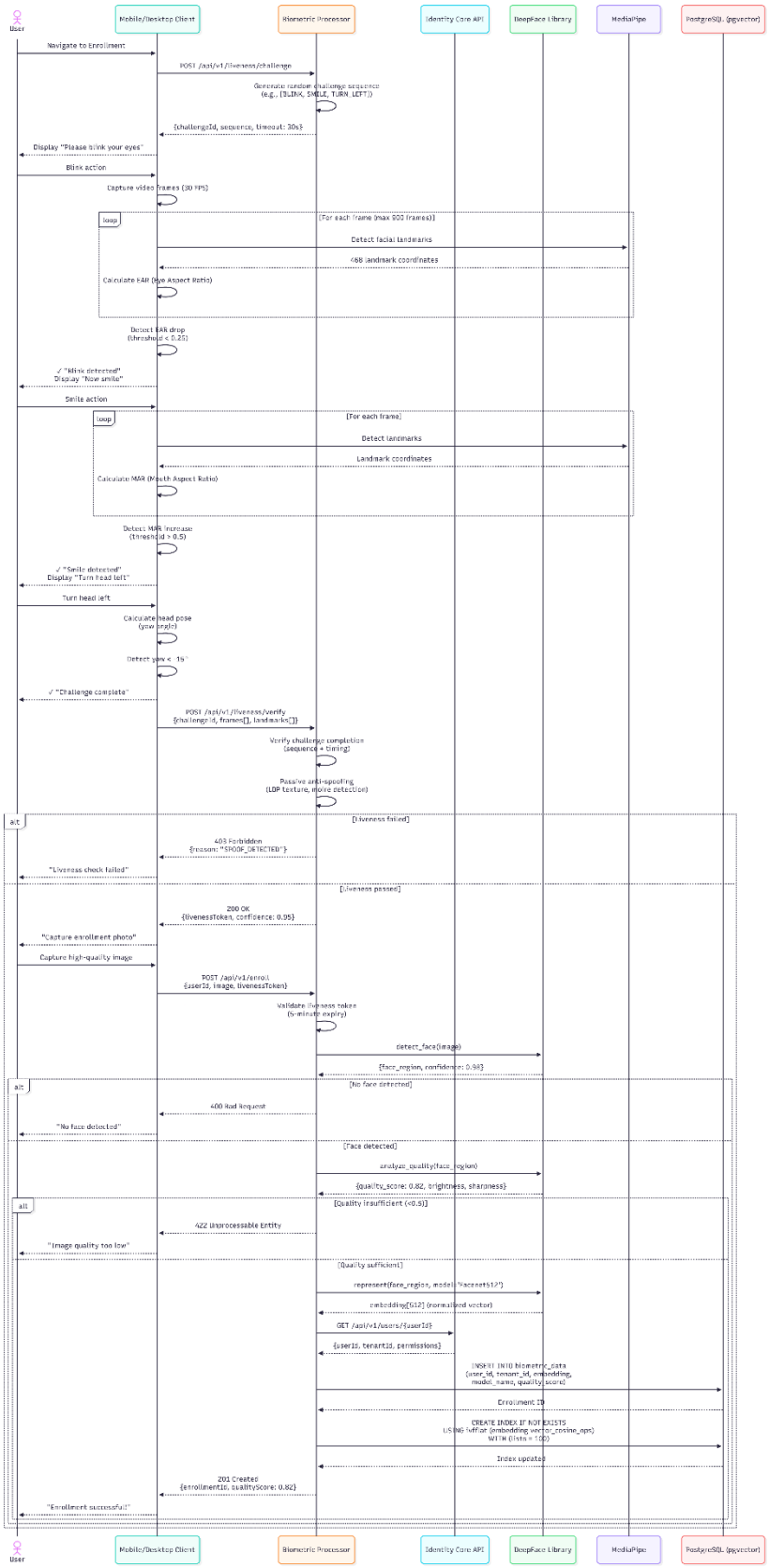


Figure 5: Biometric Enrollment with Liveness Sequence

4.1.4.3 Face Search (1:N Identification) Sequence

Sequence diagram illustrating the face verification workflow in FIVUCSAS system. The process includes: face image capture, tenant authorization validation, face detection with DeepFace, FaceNet512 embedding generation, Redis cache lookup for recent searches, PostgreSQL vector similarity search using pgvector's IVFFlat index with cosine similarity threshold, result ranking by similarity score, cache storage, and identity verification response.

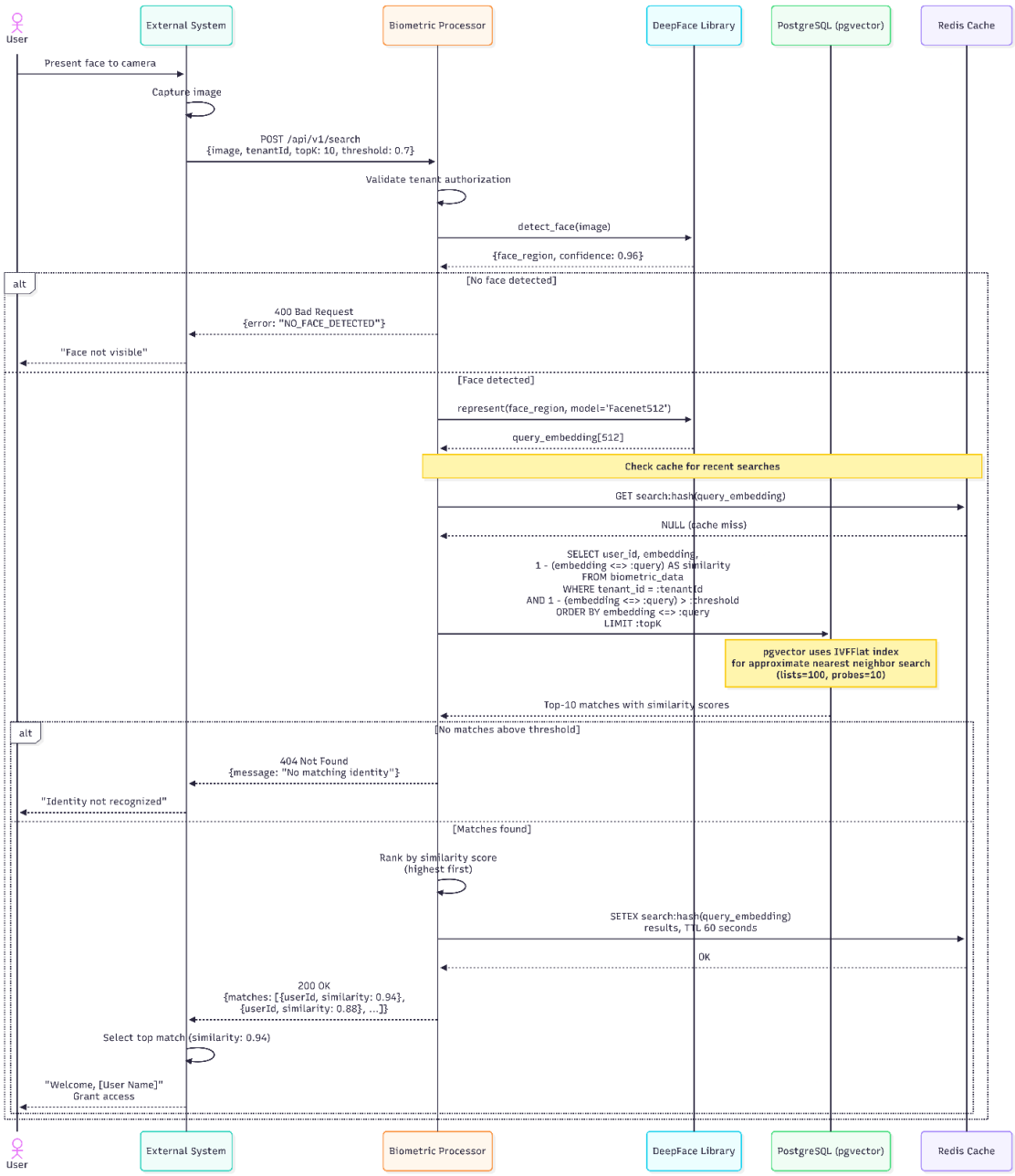


Figure 6: Face Search (1:N Identification) Sequence

4.2 Class and ER Diagrams

4.2.1 Domain Model - Core Entities

UML class diagram illustrating the core domain model of FIVUCSAS system. The model demonstrates tenant-based data isolation, role-based access control, and comprehensive biometric verification tracking with audit trail support.

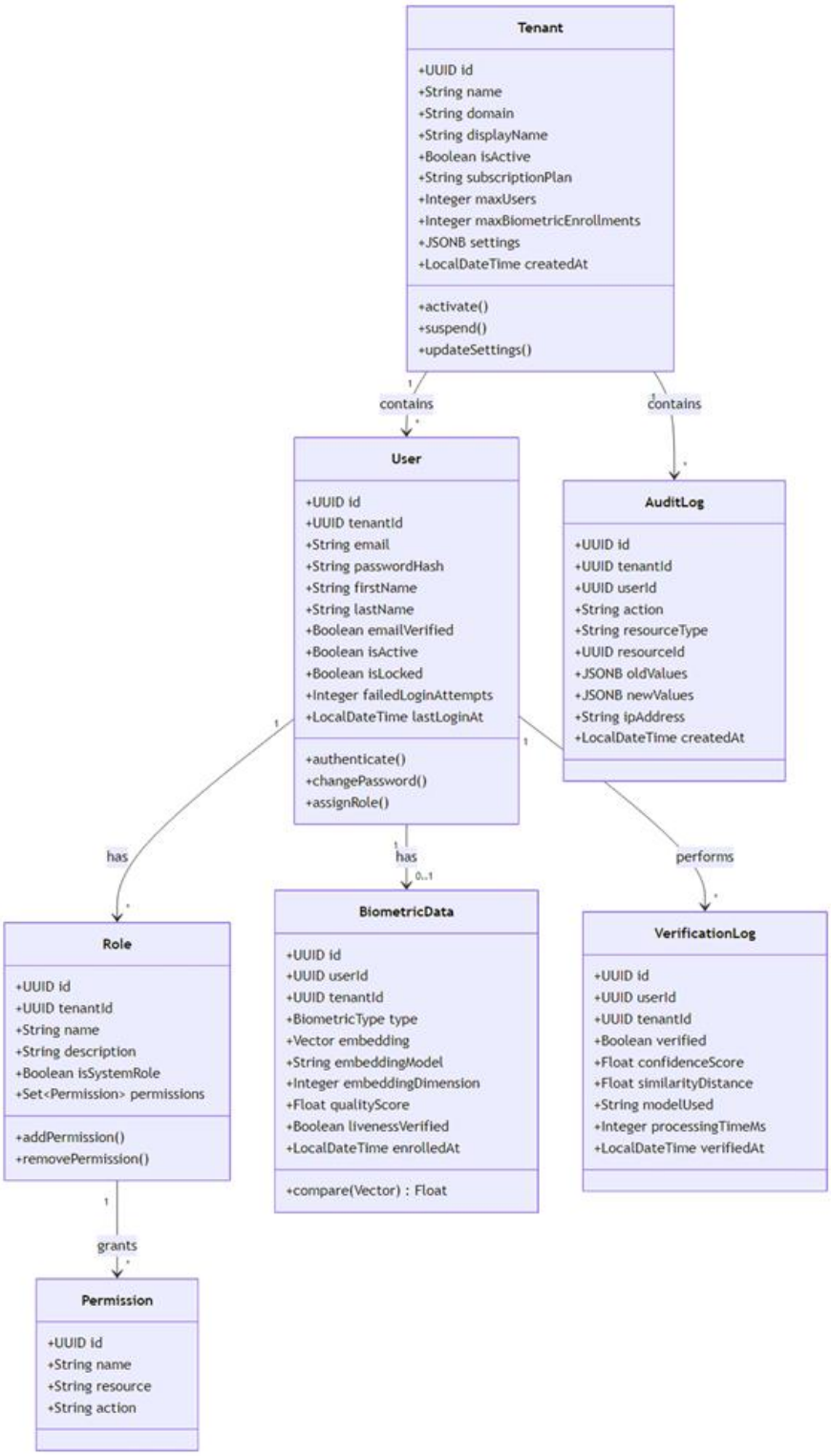


Figure 7: Core Domain Model - Multi-Tenant Identity and Biometric Verification

4.2.2 Entity Relationship Diagram

Two-part entity-relationship diagram showing: (1) Core IAM schema with multi-tenant user management, RBAC through roles and permissions, and session/token management; (2) Biometric verification schema with pgvector-based embedding storage, liveness detection logs, security event monitoring, and comprehensive audit

trails. Both schemas implement tenant isolation and include standard audit timestamps.

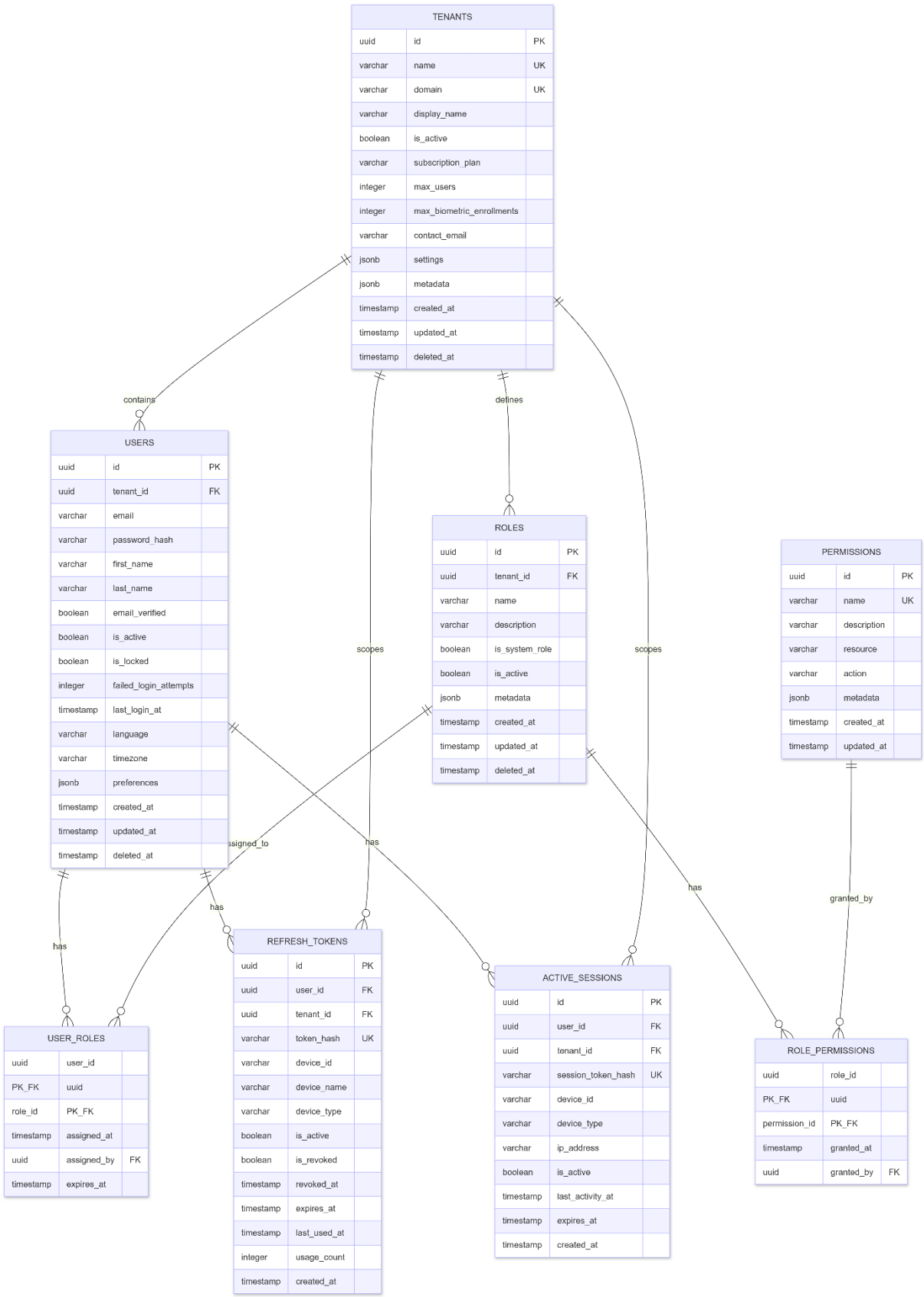


Figure 8: Core Identity and Access Management Entity-Relationship Diagram

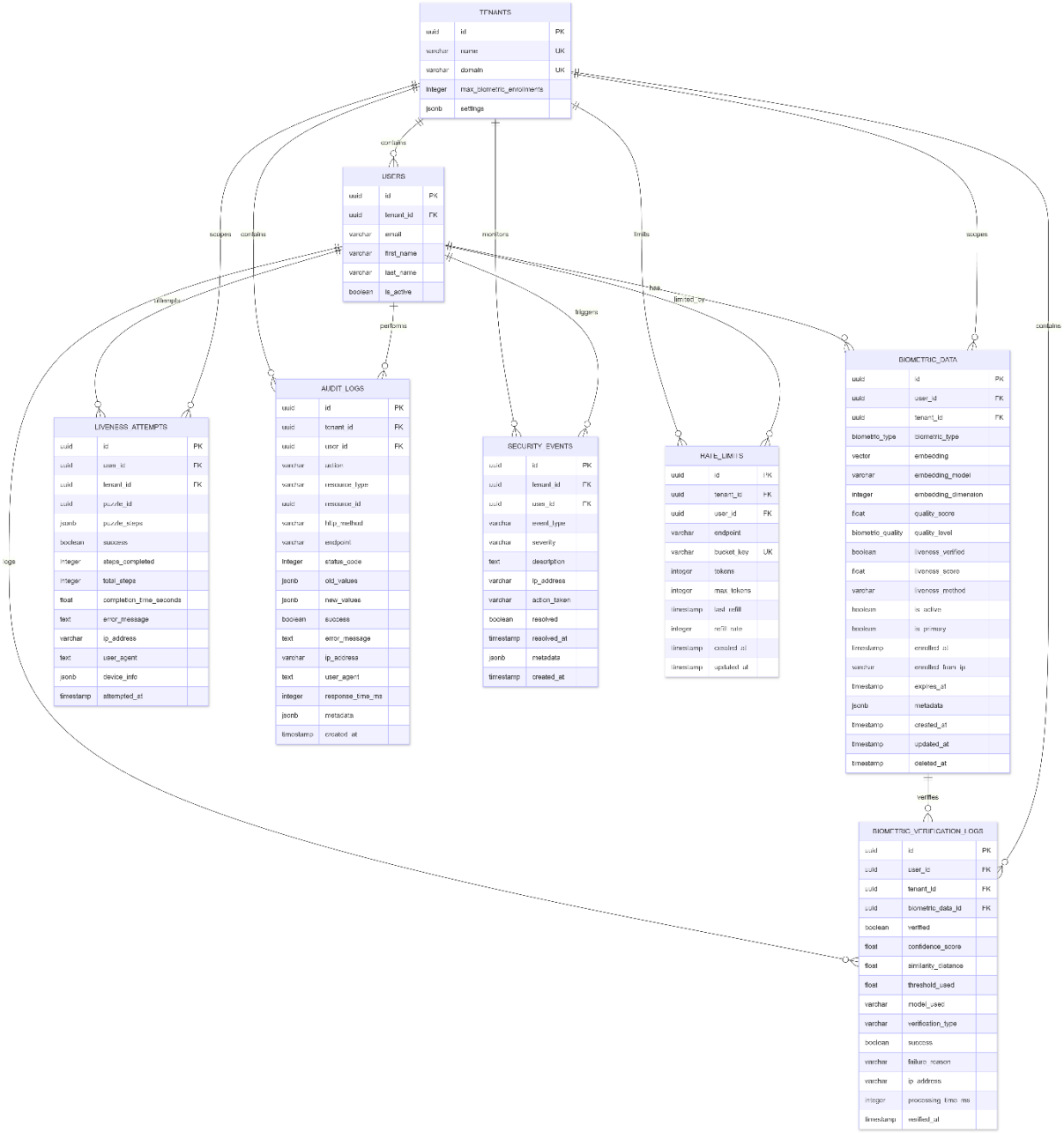


Figure 9: Biometric Verification and Security Monitoring Entity-Relationship Diagram

4.3 User Interface Design

4.3.1 Web Admin Dashboard

The web admin dashboard is built with React 18 and Material-UI v5, providing a responsive interface for tenant administrators.

Implemented Pages:

Page	Route	Description

Login	/login	Email/password authentication with remember me
Dashboard	/dashboard	Analytics overview, enrollment statistics, verification trends
Users	/users	User management with CRUD operations, role assignment
Tenants	/tenants	Tenant list and configuration (super admin only)
Enrollments	/enrollments	Biometric enrollment records with quality scores
Audit Logs	/audit-logs	Searchable, filterable audit trail
Settings	/settings	Tenant configuration, thresholds, rate limits

Table 5: Web Application Routes

4.3.2 Mobile Application (Android)

The Android application uses Kotlin Multiplatform with Compose Multiplatform UI.

Implemented Screens:

Screen	Description
Login	Email/password authentication
Register	New user registration
Home	User profile, enrollment status
Enroll	Face capture with quality feedback, liveness challenge
Verify	Face verification with real-time feedback

Table 6: Mobile Application Routes

4.3.3 Desktop Application

Desktop application targets Windows, Linux, and macOS using Kotlin Multiplatform.

Modes:

1. Kiosk Mode: Self-service terminal for enrollment and verification
2. Admin Dashboard: Desktop version of web admin functionality

Technical Implementation:

- JavaCV for camera access (cross-platform)
- Compose Desktop for UI
- Same shared code as mobile (95% reuse)

4.4 Test Plan

The testing strategy for the FIVUCSAS platform adopts a multi-layered approach, ensuring robust functionality across microservices, security compliance, and high fidelity in biometric verification. The plan emphasizes automated testing for continuous integration and specialized validation protocols for the "Biometric Puzzle" liveness detection.

4.4.1 Test Strategy

Test Level	Scope	Tools	Coverage Target
Unit Tests	Individual classes and functions	JUnit 5, pytest, Vitest	> 70%
Integration Tests	Service interactions, database	Testcontainers, pytest	API contracts
E2E Tests	Full user flows	Playwright, Espresso	Critical paths
Performance Tests	Load and stress	k6, Locust	NFR targets
Security Tests	Vulnerability scanning	OWASP ZAP, Snyk	OWASP Top 10

Table 7: Test Strategy Overview

4.4.2 Test Cases - Identity Core API

Test ID	Category	Description	Expected Result
TC-AUTH-001	Authentication	Valid login with correct credentials	200 OK, JWT tokens returned
TC-AUTH-002	Authentication	Login with invalid password	401 Unauthorized
TC-AUTH-003	Authentication	Login to locked account	423 Locked
TC-AUTH-004	Authentication	Token refresh with valid refresh token	200 OK, new tokens
TC-AUTH-005	Authentication	Token refresh with revoked token	401 Unauthorized
TC-USER-001	User Management	Create user with valid data	201 Created
TC-USER-002	User Management	Create user with duplicate email	409 Conflict
TC-USER-003	User Management	Update user profile	200 OK
TC-RBAC-001	Authorization	Access with sufficient permissions	200 OK
TC-RBAC-002	Authorization	Access with insufficient permissions	403 Forbidden
TC-TENANT-001	Multi-tenancy	User cannot access other tenant data	404 Not Found

Table 8: Identity Core API Test Cases

4.4.3 Test Cases - Biometric Processor

Test ID	Category	Description	Expected Result
TC-BIO-001	Face Detection	Valid face image	Face detected with coordinates
TC-BIO-002	Face Detection	Image without face	No face detected error
TC-BIO-003	Quality	High quality image	Score > 0.8
TC-BIO-004	Quality	Blurry image	Score < 0.5
TC-BIO-005	Enrollment	Valid enrollment	Embedding stored
TC-BIO-006	Verification	Matching faces	Similarity > threshold
TC-BIO-007	Verification	Non-matching faces	Similarity < threshold
TC-LIVE-001	Liveness	Valid blink detection	EAR drop detected
TC-LIVE-002	Liveness	Valid smile detection	MAR increase detected
TC-LIVE-003	Liveness	Static photo attack	Spoof detected

Table 9: Biometric Processor Test Cases

4.4.4 Current Test Coverage

Service	Test Files	Test Cases	Coverage
Identity Core API	29	156	72%
Biometric Processor	18	89	68%
Web App	10	45	65%

Table 10: Current Test Coverage

4.4.5 Test Timeline

This subsection provides estimated calendar time required for testing tasks.

Task No	Test Task / Milestone	Deadline	Status
Phase 1: Unit Testing			
T-1.1	Identity Core - Unit Tests	Dec 22, 2025	Complete
T-1.2	Biometric Processor - Unit Tests	Dec 26, 2025	Complete
T-1.3	Liveness Detection - Unit Tests	Dec 29, 2025	Complete
T-1.4	Client Apps - Unit Tests	Jan 2, 2026	Complete
Phase 2: Integration Testing			
T-2.1	Identity Core ↔ PostgreSQL	Jan 5, 2026	Complete

T-2.2	Biometric Processor ↔ pgvector	Jan 9, 2026	Complete
T-2.3	Identity Core ↔ Redis	Jan 12, 2026	Complete
T-2.4	Biometric Processor ↔ DeepFace	Jan 16, 2026	Complete
Phase 3: End-to-End Testing			
T-3.1	User Registration → Enrollment Flow	Jan 26, 2026	Pending
T-3.2	Liveness → Verification → Access Flow	Feb 2, 2026	Pending
T-3.3	Multi-tenant Isolation Tests	Feb 9, 2026	Pending
T-3.4	Mobile App Integration Tests	Feb 16, 2026	Pending
Phase 4: Performance Testing			
T-4.1	API Load Testing (500 RPS)	Feb 23, 2026	Pending
T-4.2	Vector Search Benchmark	Mar 2, 2026	Pending
T-4.3	Liveness Detection Latency	Mar 9, 2026	Pending
Phase 5: Security Testing			
T-5.1	OWASP Vulnerability Scan	Mar 23, 2026	Pending
T-5.2	Authentication Bypass Tests	Mar 30, 2026	Pending
T-5.3	Spoofing Attack Tests	Apr 6, 2026	Pending
Phase 6: Final Validation			
T-6.1	Full Regression Suite	Apr 20, 2026	Pending
T-6.2	Bug Fixes & Re-testing	May 4, 2026	Pending
T-6.3	Documentation & Test Report	May 15, 2026	Pending

Table 11: Test Timeline

5. Software Architecture

The software architecture of FIVUCSAS is designed as a scalable, cloud-native ecosystem that prioritizes modularity, high-performance biometric processing, and strict multi-tenant isolation. By adopting a microservices-based approach, the system decouples administrative identity management from compute-intensive machine learning tasks, ensuring that each component can scale independently based on demand. The architecture is organized into distinct logical layers—ranging from cross-platform client interfaces to a robust data layer optimized for vector similarity search all orchestrated through a centralized API Gateway that ensures secure and efficient request routing across the entire platform.[7][16]

5.1 Architectural Style

FIVUCSAS employs a microservices architecture with Hexagonal Architecture (Ports & Adapters) within each service, ensuring:

- Separation of Concerns: Business logic isolated from infrastructure
- Testability: Domain logic testable without external dependencies
- Flexibility: Easy to swap implementations (e.g., database, ML models)
- Scalability: Services scale independently based on load [7]

Within FIVUCSAS, each microservice (e.g., Identity Core API and Biometric Processor API) independently applies the Hexagonal Architecture, where external interfaces such as REST APIs and message consumers interact with application use cases through defined ports. This approach ensures that core domain logic remains independent of frameworks, databases, and communication mechanisms. [7]

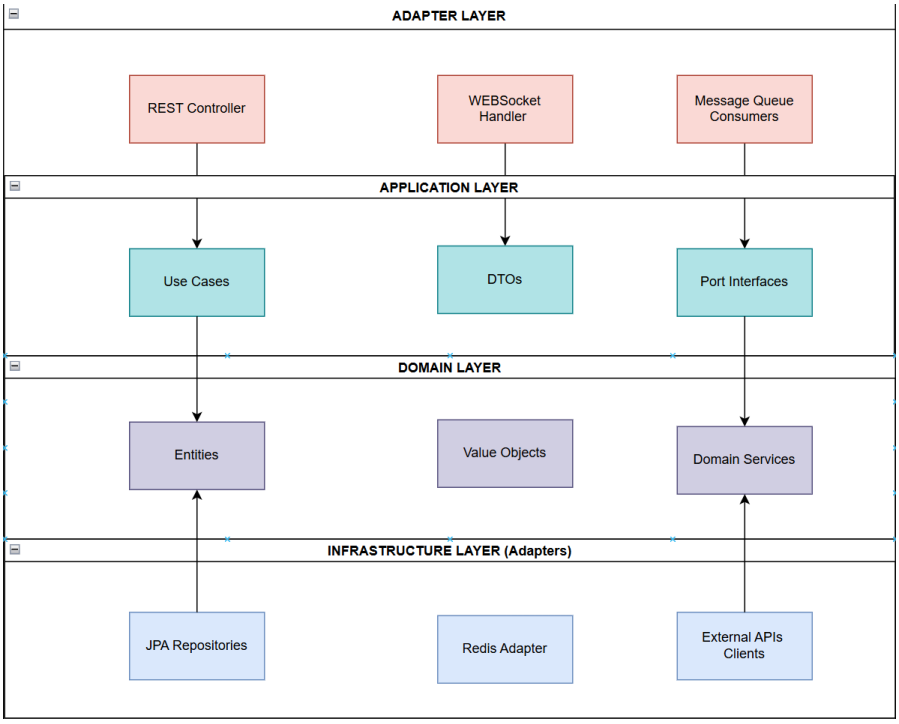


Figure 10: Layers

5.2 Component Architecture

5.2.1 High-Level Architecture

This figure illustrates the distributed microservices model of the platform, detailing the hierarchical flow of data from diverse cross-platform clients through an **NGINX API Gateway** to the core application microservices the **Identity Core API** and the **Biometric Processor** while highlighting the unified data layer and specialized machine learning integration. [16]

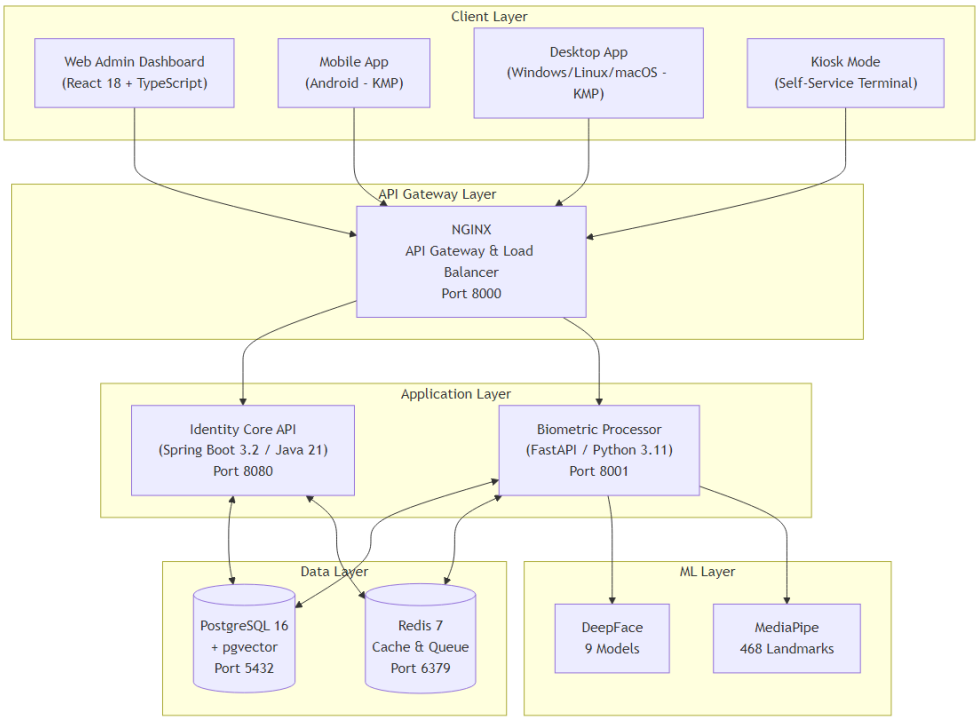


Figure 11: High-Level System Architecture Overview

5.2.2 Identity Core API Components

The Identity Core API is structured according to the Hexagonal Architecture (Ports and Adapters) to ensure a clear separation between business logic, external interfaces, and infrastructure concerns. Incoming requests are handled through web adapters, while persistence and messaging are abstracted via outbound ports. This design improves testability, enables independent evolution of infrastructure components, and supports multi-tenant scalability and security requirements. [15]

Technology Stack:

- Spring Boot 3.2.0
- Java 21
- Spring Data JPA
- Spring Security
- JJWT for JWT handling
- Bucket4j for rate limiting
- Flyway for migrations

Package Structure:

```
com.fivucsas.identity/
├── adapter/
│   ├── in/
│   │   └── web/           # REST Controllers (8 controllers)
│   └── out/
│       ├── persistence/    # JPA Repositories
│       └── messaging/      # Redis messaging
├── application/
```

```
│   ├── port/
│   │   ├── in/            # Use case interfaces (13 use cases)
│   │   └── out/           # Repository interfaces
│   ├── service/           # Use case implementations
│   └── dto/               # Data Transfer Objects
├── domain/
│   ├── model/             # Entities (10 entities)
│   └── service/           # Domain services
└── infrastructure/
    ├── config/            # Spring configuration
    ├── security/          # Security configuration
    └── persistence/       # JPA entities and configs
```

Key Controllers:

Controller	Endpoints	Description
AuthController	/api/v1/auth/*	Login, logout, token refresh
UserController	/api/v1/users/*	User CRUD operations
TenantController	/api/v1/tenants/*	Tenant management
RoleController	/api/v1/roles/*	Role and permission management
BiometricController	/api/v1/biometric/*	Proxy to biometric processor
AuditController	/api/v1/audit/*	Audit log queries
HealthController	/health/*	Health checks
SettingsController	/api/v1/settings/*	Configuration management

Table 12: *Identity Core API Key Controllers and Endpoint Definitions*

*This table **outlines** the primary REST controllers within the **Identity Core API**, mapping their respective base URL patterns to their specific administrative and operational responsibilities.*

5.2.3 Biometric Processor Components

The Biometric Processor is designed as a compute-intensive microservice responsible for all face analysis and biometric operations. It follows the same Hexagonal Architecture principles, where API routes act as inbound adapters, application use cases encapsulate biometric workflows, and infrastructure adapters wrap machine learning models and external dependencies. This separation allows independent scaling, efficient GPU utilization, and seamless integration of new biometric models without affecting higher-level services. [15]

Package Structure:

```
app/
├── api/
│   └── routes/           # 19 route modules
│       ├── health.py
│       ├── enrollment.py
│       ├── verification.py
│       ├── search.py
│       ├── liveness.py
```

```
│       ├── quality.py
│       ├── admin.py
│       ├── analytics.py
│       ├── config.py
│       ├── detection.py
│       ├── embedding.py
│       ├── landmarks.py
│       ├── processing.py
│       ├── proctoring.py (WebSocket)
│       └── ... (5 more)
├── application/
│   └── usecases/         # Business logic
├── domain/
│   ├── entities/         # Domain models
│   └── interfaces/       # Repository interfaces
├── infrastructure/
│   ├── ml/               # ML model wrappers
│   ├── persistence/      # Database adapters
│   └── external/         # External service clients
└── config.py             # 552-line configuration
```

Technology Stack:

- FastAPI 0.104+
- Python 3.11+
- DeepFace 0.0.79+
- MediaPipe 0.10+
- SQLAlchemy
- asyncpg for async PostgreSQL
- OpenCV

API Endpoint Summary (46+ endpoints):

Category	Endpoints	Description
Health	3	Readiness, liveness, model status
Enrollment	6	Enroll, re-enroll, delete, status
Verification	5	1:1 verify, verify with liveness
Search	4	1:N search, batch search
Liveness	8	Challenge generation, verification, puzzle steps
Quality	4	Assess, batch assess, metrics
Detection	5	Detect faces, landmarks, attributes
Embedding	4	Generate, compare, batch generate
Analytics	4	Usage stats, accuracy metrics
Admin	3	Model management, cache control

Table 13: *Functional Categorization and Distribution of Biometric Processor API Endpoints*

This table provides a comprehensive summary of the ***Biometric Processor's*** *interface, highlighting a diverse set of over 46 specialized endpoints—ranging from real-time* ***Liveness*** *challenges to deep-learning* ***Search*** *operations that collectively support the core biometric logic and proctoring capabilities of the FIVUCSAS platform.*

Supported Face Recognition Models:

Model	Dimensions	Speed	Accuracy
VGG-Face	2,622	Medium	Good
Facenet	128	Fast	Good
Facenet512	512	Medium	Excellent
OpenFace	128	Fast	Moderate
DeepFace	4,096	Slow	Good
DeepID	160	Fast	Moderate
ArcFace	512	Medium	Excellent
Dlib	128	Fast	Good
SFace	128	Fast	Good

Table 14: *Performance Matrix of Supported Face Recognition Models*

The following matrix evaluates the nine distinct face recognition models supported by the system, illustrating the specific trade-offs between ***Dimensions****,* ***Inference Speed****, and* ***Predictive Accuracy****; this multi-model flexibility allows tenants to optimize the system for either high-speed edge verification (e.g., SFace) or high-assurance identity search (e.g., ArcFace).*

5.3 Data Architecture

5.3.1 Database Design

Database: PostgreSQL 16 with pgvector extension [3]

Key Design Decisions:

3. Multi-tenancy: Shared database, shared schema with tenant_id column
4. Row-Level Security: Enforced at application layer
5. Soft Deletes: deleted_at timestamp for audit compliance
6. Vector Storage: pgvector with IVFFlat indexing for similarity search [3]

Migration History (Flyway):

Version	Description	Tables Created
V1	Tenants	tenants
V2	Users	users
V3	RBAC	roles, permissions, rolepermissions, userroles
V4	Biometrics	biometricdata, livenessattempts, biometricverificationlogs
V5	Audit & Sessions	auditlogs, refreshtokens, activesessions, passwordhistory, security_events

V6	Refresh Tokens	Token enhancements
V7	Performance	18 composite indexes
V8	Audit Enhancements	Additional audit fields
V9	Rate Limiting	rate_limits table

Table 15: *Database Migration History and Schema Evolution using Flyway*

*This table outlines the structured evolution of the **FIVUCSAS** data architecture across nine distinct migration versions, detailing the incremental establishment of core system entities from initial multi-tenant tenant and user structures to advanced **RBAC** frameworks, biometric vector storage, and optimized performance indexing.*

Vector Index Configuration:

```
-- IVFFlat index for approximate nearest neighbor search
CREATE INDEX idx_biometric_embedding_ivfflat
    ON biometric_data
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100)
    WHERE deleted_at IS NULL AND is_active = TRUE;
```

5.3.2 Caching Strategy

Redis Usage:

Purpose	Key Pattern	TTL
Session Cache	session:{userid}:{tokenhash}	15 min
Rate Limit Buckets	ratelimit:{tenantid}:{user_id}:{endpoint}	1 min
User Cache	user:{tenantid}:{userid}	5 min
Permission Cache	permissions:{user_id}	5 min

Table 16: *Redis Caching Strategy for Session Management and Performance Optimization*

*This table details the high-speed caching layer of the system, specifying the key-naming patterns and Time-To-Live (TTL) parameters for volatile data such as **Session Cookies**, **Rate Limit Buckets**, and **User Permissions**; this strategy is essential for minimizing primary database latency and ensuring the system can handle up to 500 requests per second as specified in the non-functional requirements.*

5.4 Deployment Architecture

5.4.1 Docker Compose Development Environment

This figure illustrates the isolated network topology of the **FIVUCSAS** platform, detailing the orchestration of frontend, backend, and infrastructure containers within a unified Docker network. [8]

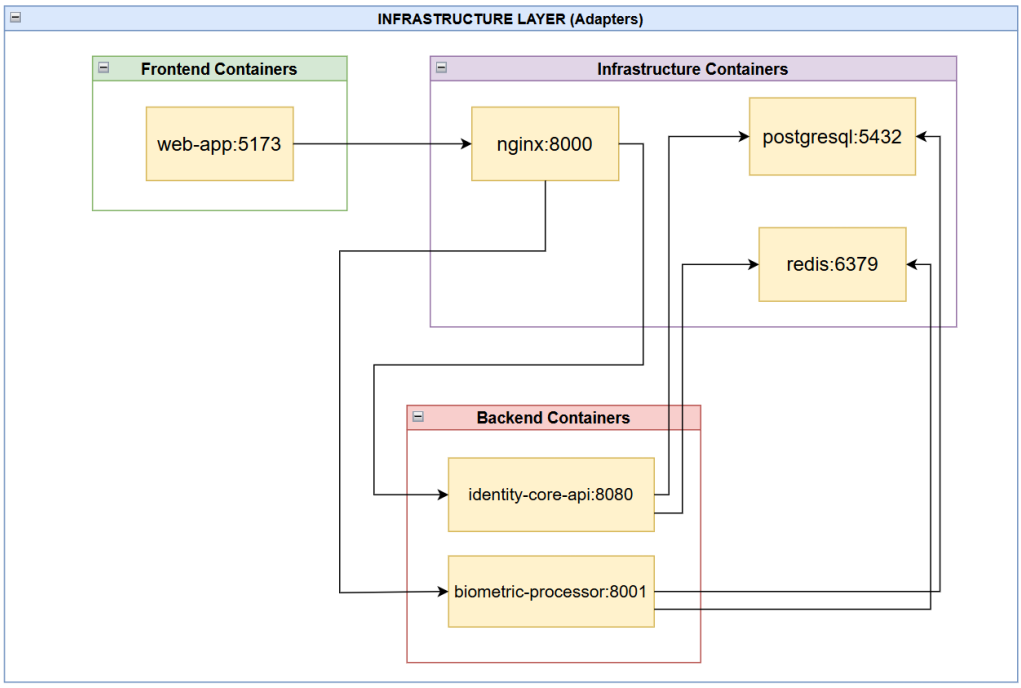


Figure 12: Docker Compose Development Environment and Containerized Service Topology

Container Specifications:

This table defines the containerized foundation of the platform, specifying the lightweight base images such as **Eclipse Temurin** for the Java-based Identity Core and **Python-slim** for the Biometric Processor alongside their respective memory allocations to ensure a stable and resource-efficient development environment. [8][14][16]

Service	Base Image	Resources (Dev)
identity-core-api	eclipse-temurin:21-jre	512MB RAM
biometric-processor	python:3.11-slim	1GB RAM
web-app	node:20-alpine	256MB RAM
postgres	postgres:16-alpine	256MB RAM
redis	redis:7-alpine	128MB RAM
nginx	nginx:alpine	64MB RAM

Table 17: *Docker Container Specifications and Resource Allocation for Development*

5.4.2 Environment Configuration

The secure and modular operation of the **FIVUCSAS** platform relies on a set of externalized environment variables, which decouple the application code from infrastructure-specific settings. As illustrated in the configuration reference, these variables manage critical system parameters:

```
# Database
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=fivucsas
POSTGRES_USER=fivucsas
POSTGRES_PASSWORD=<secure-password>

# Redis
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=<secure-password>

# JWT
JWT_SECRET=<256-bit-key>
JWT_ACCESS_TOKEN_EXPIRY=900
JWT_REFRESH_TOKEN_EXPIRY=604800

# Biometric
DEFAULT_FACE_MODEL=VGG-Face
SIMILARITY_THRESHOLD=0.6
QUALITY_THRESHOLD=0.5
```

6. Tasks Accomplished

6.1 Current State of the System

FIVUCSAS platform provides an integrated facial analysis framework that combines modern biometric techniques with a modular and scalable software architecture. The system brings together face detection, face recognition, liveness detection, and quality assessment components into a unified processing pipeline, establishing a solid experimental foundation for secure identity verification applications.

The facial analysis pipeline begins with a **face detection module** supporting robust multi-face detection through configurable backend engines. For each detected face, precise bounding box extraction is performed, followed by face alignment and normalization as illustrated in Figure 13. These preprocessing steps aim to ensure consistency across varying acquisition conditions and to improve the reliability of downstream biometric processing.

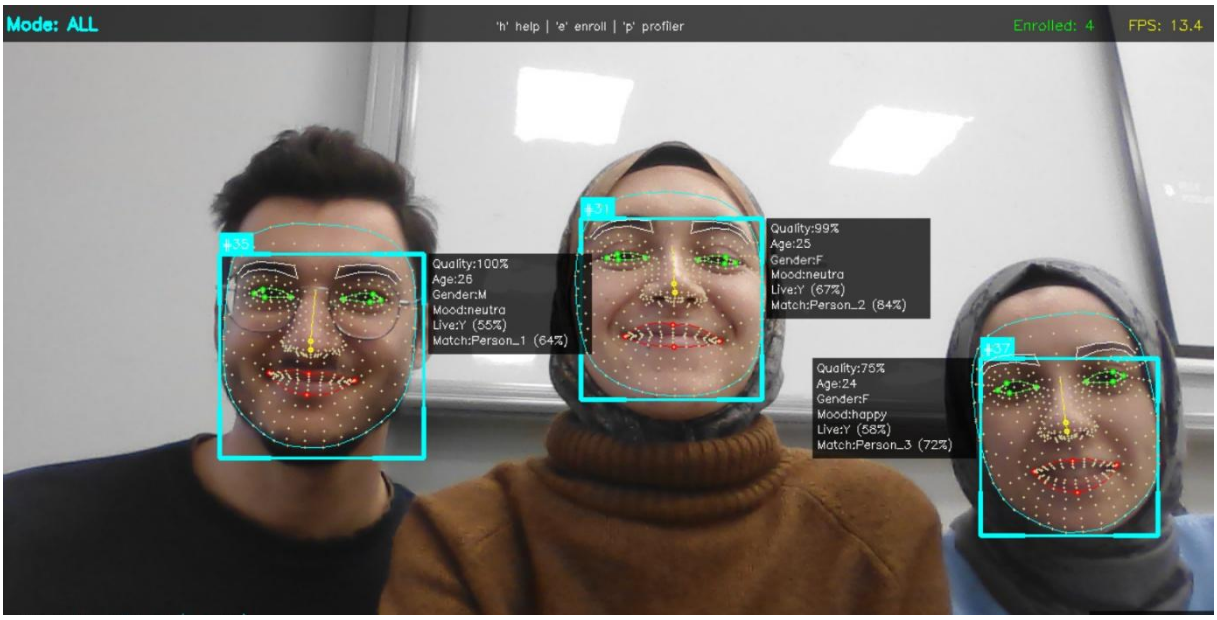


Figure 13: Multi-face detection, landmark extraction, and real-time identity matching results

The **face recognition component** processes normalized facial images to generate high-dimensional biometric embeddings. These embeddings are stored in a **multi-tenant PostgreSQL database enhanced with pgvector**, enabling efficient similarity search and identity comparison. The subsystem supports embedding comparison as well as batch processing, allowing the system to be evaluated under both real-time and higher-volume identification scenarios.

To address spoofing and presentation attacks, the platform incorporates a **multi-layer liveness detection strategy** that combines active and passive approaches. The active liveness mechanism follows a **Biometric Puzzle** concept implemented using MediaPipe, where users are required to complete randomized challenge sequences. These challenges include blink detection based on the Eye Aspect Ratio (EAR), smile detection using the Mouth Aspect Ratio (MAR), and head pose tracking across yaw, pitch, and roll axes. This challenge-response structure is designed to reduce susceptibility to replay-based attacks. Complementing this, the passive liveness layer analyzes intrinsic image characteristics without explicit user interaction (Figure 14), employing texture analysis via Local Binary Patterns (LBP), color distribution analysis, moiré pattern detection, and frequency-domain analysis to identify presentation artifacts.

In parallel, a dedicated **quality assessment module** evaluates facial samples prior to biometric processing. This module assesses brightness levels, sharpness and blur metrics, facial pose compliance, occlusion presence, and minimum resolution requirements, ensuring that unsuitable samples are filtered out before reaching subsequent processing stages.

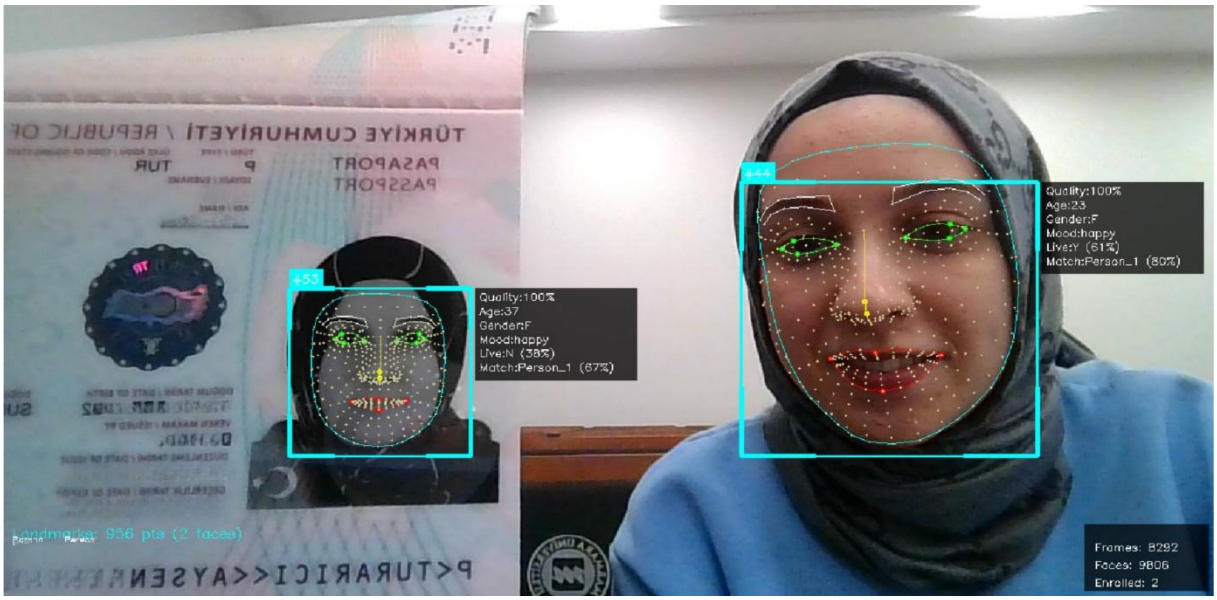


Figure 14: Presentation attack scenario using printed facial images and passive liveness analysis

Beyond facial biometrics, the system includes a specialized **NFC integration module** capable of authenticating and extracting data from multiple card types as demonstrated in Figure 15, including Turkish National Identity Cards, ICAO 9303–compliant biometric passports, driver's licenses, Marmara University student identification cards, and academic staff cards. This integration enables the exploration of combined physical token-based authentication and digital biometric verification workflows.

Figure 15:Passport detection and classification within the NFC-enabled card authentication module

The overall system is supported by a **production-oriented backend infrastructure**, exposing more than 46 RESTful endpoints documented via Swagger/OpenAPI. The architecture follows **Hexagonal Architecture principles**, providing a clear separation between core biometric logic and infrastructure components, and facilitating maintainability, extensibility, and iterative development.

In its current state, the FIVUCSAS platform represents a stable and functional experimental system in which the core biometric pipeline and supporting services have been implemented and validated in controlled environments. Ongoing efforts focus on extended field testing, performance evaluation, and robustness analysis under real-world conditions, guiding the continued evolution of the system in subsequent development phases.

6.2 Task Log

Meeting #1

Date: 12.09.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Project topic selection and team formation

Decisions and Notes:

- Final Engineering Project topic (FIVUCSAS) was selected and approved.
- Team roles and responsibilities were assigned.
- Project scope and core scenarios were defined.
- Initial literature sources on face recognition and biometric systems were identified.

Meeting #2

Date: 26.09.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Architecture design and technology selection

Decisions and Notes:

- System architecture and microservice structure were designed.
- Backend, frontend, and database technologies were selected.
- Face recognition and liveness detection literature was reviewed.

- Commercial biometric solutions were analyzed for comparison.

Meeting #3

Date: 10.10.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Database design and multi-tenancy strategy

Decisions and Notes:

- Database schema and multi-tenancy strategy were finalized.
- Biometric data storage using face embeddings was defined.
- Vector similarity search techniques were evaluated.
- Literature on biometric template protection was reviewed.

Meeting #4

Date: 24.10.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: API design and security architecture review

Decisions and Notes:

- Identity Core API design and security architecture were reviewed.
- Authentication and authorization mechanisms were finalized.
- Secure biometric authentication studies were analyzed.
- API versioning and documentation strategy were defined.

Meeting #5

Date: 07.11.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Biometric processing prototype implementation

Decisions and Notes:

- DeepFace-based biometric processing prototype was implemented.
- Biometric enrollment and verification workflows were defined.
- Image quality assessment criteria were established.
- Performance optimization strategies were discussed.

Meeting #6

Date: 14.11.2025

Location: Marmara University, Engineering Faculty

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Mid-semester demo and progress evaluation

Decisions and Notes:

- Mid-semester system demo was conducted.
- Project progress was evaluated against the timeline.
- Integration and performance risks were identified.
- Accuracy metrics evaluation approach was defined.

Meeting #7

Date: 28.11.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Liveness detection algorithm evaluation

Decisions and Notes:

- Biometric Puzzle liveness detection algorithm evaluated.
- Active and passive liveness detection methods were combined.
- MediaPipe facial landmark detection was integrated.
- Mobile liveness processing strategy was defined.

Meeting #8

Date: 05.12.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: UI/UX design review across platforms

Decisions and Notes:

- Web dashboard feature set was discussed.
- Mobile application screen flows were discussed.
- Desktop application modes were defined.
- UI/UX consistency across platforms was established.

Meeting #9

Date: 12.12.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Service integration and API contracts

Decisions and Notes:

- Identity Core and Biometric Processor integration strategy was completed.
- API contracts and standardized error responses were finalized.
- Integration and end-to-end testing plan was defined.

Meeting #10

Date: 26.12.2025

Location: Online (Microsoft Teams)

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Documentation review and spring planning

Decisions and Notes:

- Documentation was reviewed and revised.
- Completed system components were demonstrated.
- Performance metrics were identified.
- Spring semester focus areas were clarified.

Meeting #11

Date: 09.01.2026

Location: Marmara University, Engineering Faculty

Period: [UPDATE HOURS]

Attendees: All team members, Advisor

Objectives: Fall semester retrospective and ADD review

Decisions and Notes:

- Fall semester retrospective was conducted.
- Spring semester goals and priorities were defined.
- Service integration tasks were prioritized.
- ADD document requirements were reviewed with the advisor.
- Mandatory diagrams and performance metrics were clarified.

6.3 Task Plan

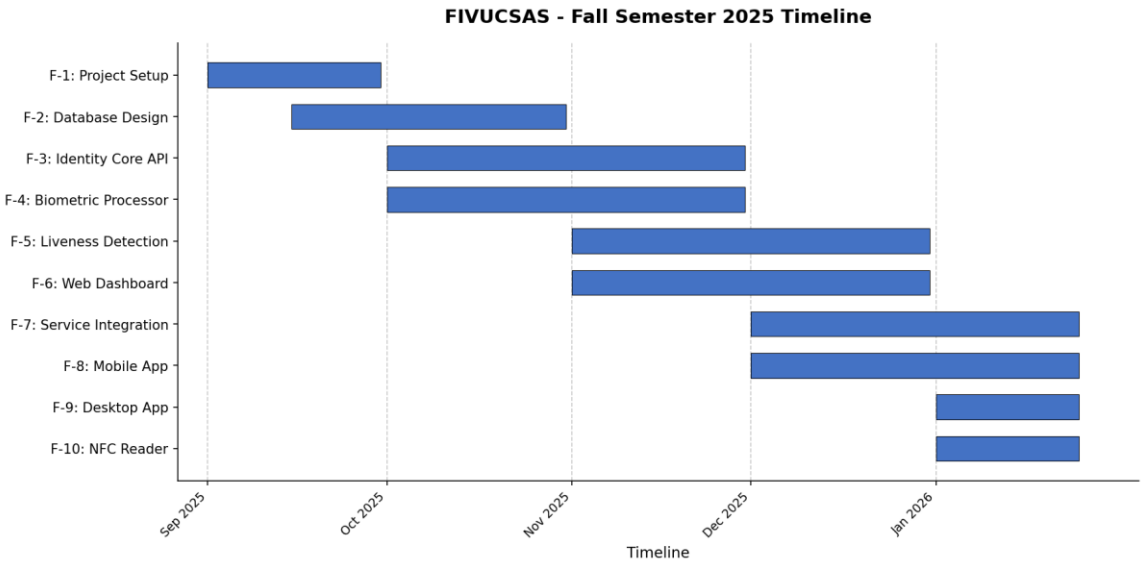
6.3.1 Fall Semester Timeline

Task No.	Task Description	Sep	Oct	Nov	Dec	Jan
F-1	**Project Initialization & Setup** *Git repository, CI/CD Pipeline, Literature Review*					
F-2	**Database Schema Design** *ER Diagram, Flyway Migrations, pgvector Setup*					
F-3	**Identity Core - Base Implementation** *User Registration, JWT Auth*					
F-4	**Biometric Processor - Core API** *Face Detection, Embedding, Quality Analysis*					
F-5	**Liveness Detection Algorithm** *Biometric Puzzle, MediaPipe Integration*					
F-6	**Web Admin Dashboard** *React 18 UI, Shadcn/UI Components*					
F-7	**Service Integration** *Identity Core ↔ Biometric Processor*					
F-8	**Mobile App** *Service communication*					
F-9	**Desktop App** *Enrollment and Identity Verification steps*					
F-10	**NFC Reader** *Turkish eID Reader, Universal Card Detector*					

Table 18: Completed Tasks in Fall Semester

Note: Shaded cells indicate active development periods.

Figure: Fall Semester Gantt Chart

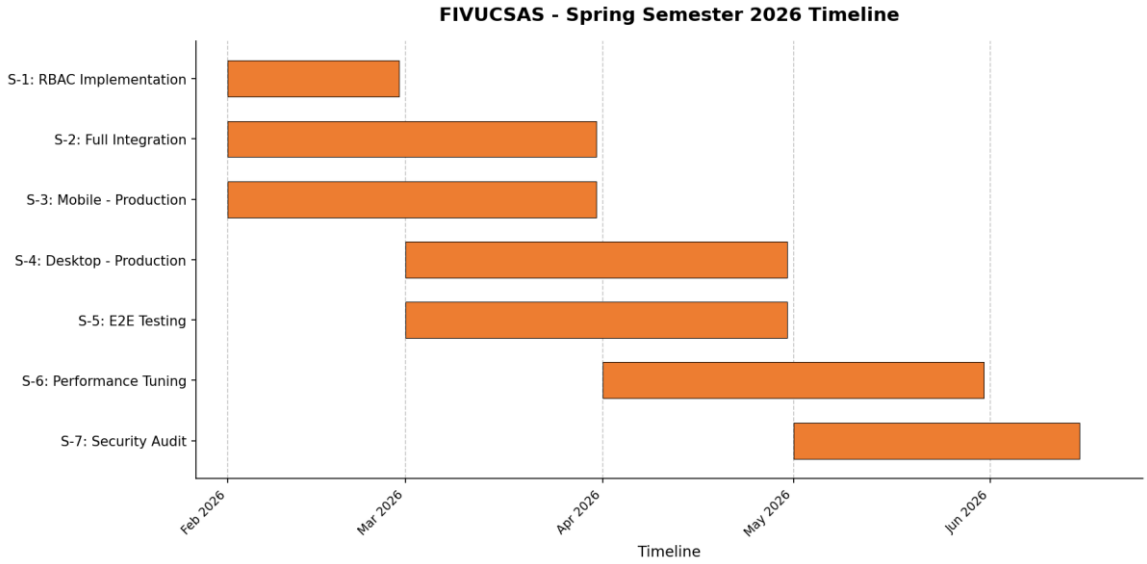


6.3.1 Spring Semester Timeline

Task No.	Task Description	Feb	Mar	Apr	May	Jun
S-1	**RBAC Implementation** *Permission enforcement, role-based access control*					
S-2	**Service Integration - Complete** *Full Identity ↔ Biometric integration, webhooks*					
S-3	**Mobile App – Production Ready** *API client, authentication flow, biometric enrollment*					
S-4	**Desktop App - Production Ready** *Admin dashboard, session management*					
S-5	**End-to-End Testing** *Playwright tests, API integration tests, mobile E2E*					
S-6	**Performance Optimization** *Load testing, caching strategy, API response time*					
S-7	**Security Audit** *Penetration testing, vulnerability fixes*					

Table 19: Planned Tasks for Spring Semester

Figure: Spring Semester Gantt Chart



REFERENCES

[1] Verizon, *2024 Data Breach Investigations Report (DBIR)*.
Available: https://www.verizon.com/business/resources/reports/dbir/, Accessed: Jan. 2026.

[2] Identity Theft Resource Center, *2023 Data Breach Report*.
Available: https://www.idtheftcenter.org/, Accessed: Jan. 2026.

[3] PostgreSQL Global Development Group, *pgvector Extension*.
Available: https://github.com/pgvector/pgvector
Accessed: Jan. 2026.

[4] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," *IEEE CVPR*, 2014.

[5] Google, *MediaPipe Face Landmark Detection*.
Available:
https://developers.google.com/mediapipe/solutions/vision/face_landmarker
Accessed: Jan. 2026.

[6] Redis Ltd., *Redis Documentation*.
Available: https://redis.io/docs/
Accessed: Jan. 2026.

[7] A. Cockburn, “Hexagonal Architecture,” *Alistair Cockburn Blog*.
Available: https://alistair.cockburn.us/hexagonal-architecture/
Accessed: Jan. 2026.

[8] Docker Inc., *Docker Documentation*.
Available: https://docs.docker.com/
Accessed: Jan. 2026.

[9] ISO/IEC 30107-3, *Information Technology – Biometric Presentation Attack Detection*.
International Organization for Standardization, 2017.

[10] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” *IEEE/CVF CVPR*, 2019.

[11] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” *IEEE CVPR*, 2015.

[12] NGINX Inc., *NGINX Official Documentation*.
Available: https://nginx.org/en/docs/
Accessed: Jan. 2026.

[13] Facebook AI Research, FAISS: A Library for Efficient Similarity Search.

Available: https://github.com/facebookresearch/faiss

Accessed: Jan. 2026.

[14] Python Software Foundation, Python Documentation.

Available: https://docs.python.org/

Accessed: Jan. 2026.

[15] Docker Inc., Docker Compose Documentation.

Available: https://docs.docker.com/compose/

Accessed: Jan. 2026.

[16] C. Richardson, Microservices Patterns.

O'Reilly Media, 2018.

[17] O.M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep Face Recognition," British Machine Vision Conference (BMVC), 2015.

[18] S.I. Serengil and A. Ozpinar, "LightFace: A Hybrid Deep Face Recognition Framework," IEEE ASYU, 2020.